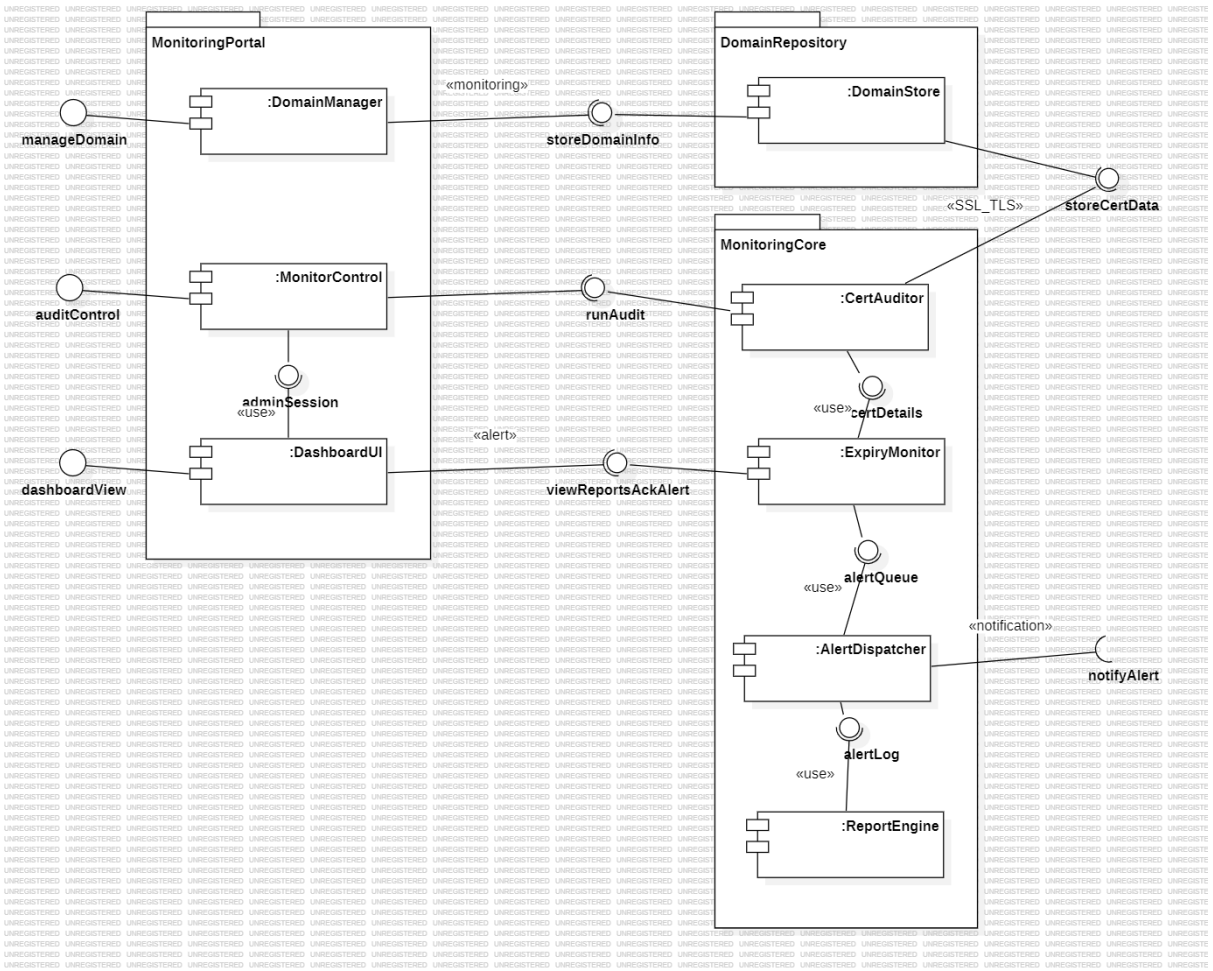
SHREYA S MATTIMADU

PES2UG24CS486

5H

SE LAB



Monitoring Portal holds everything a person interacts with directly: Domain Manager, Monitor Control, and Dashboard UI. That's why the human-facing interfaces — manageDomain, audit Control, dashboard View — all sit at this subsystem's boundary, matching the Admin and Security Officer actors from the use case diagram. Keeping the UI layer separate from the processing logic means one can change without dragging the other along with it.

Monitoring Core is where the actual work happens. CertAuditor covers both "perform WHOIS lookup" and "SSL/TLS handshake" — I merged these into one component rather than two, since they never run independently; every audit does both, so the «include» relationships collapse naturally. Expiry Monitor handles "monitor expiry and generate alerts," and it's also where the "ACK alert" extend relationship shows up, since it's the component that receives acknowledgements back from the dashboard. AlertD is

patcher takes care of the escalation ladder, and it's the one component with a required interface (notify Alert) pointing outward — escalation depends on an outside notification channel the system doesn't control, so that dependency has to be explicit. Report Engine pulls the alert history together for the dashboard. Splitting alerting from auditing this way also helps with the NFR-001 performance target (scanning 1,000 domains in under 3 minutes) — the audit path can be scaled up on its own without the alerting side getting in the way.

Domain Repository exists as its own subsystem just for Domain Store, because both the portal and the core need to read and write domain/certificate data. Giving that one place to live avoids two components stepping on each other's state, and makes it easier to lock down access to what's fairly sensitive certificate metadata.

I used ball-and-socket connectors everywhere instead of plain dependency arrows, the same way the reference Web Store diagram does — the ball marks who provides a contract, the socket marks who's just consuming it. That distinction is what lets a piece get swapped out later without a redesign: if CertAuditor were replaced with a different WHOIS/TLS library tomorrow, Monitor Control wouldn't even notice, since it only ever talks to the run Audit interface.